



윤경은

Software Engineer, ML Systems

yge0307@gmail.com
github.com/ykgstar37-lab
yge-portfolio.vercel.app

응용통계학 기반의 정량 분석 경험 위에 검색 품질 평가, ML/LLM 서빙 안정화, 시스템 최적화를 직접 맡아 온 엔지니어입니다. 최근 프로젝트들에서 **모델을 바꾸기 전에 평가 기준과 병목 구간부터 다시 정의**하고, 그 결과를 실제 구조 변경으로 연결해 왔습니다.

세 프로젝트에서 검색 품질 진단, 서빙 안정화, 비용 라우팅 최적화를 각각 맡아 **Precision +14.4%p, 정확도 37%→85%, 요청 비용 85%↓**를 만들었습니다.

EDUCATION

가천대학교 응용통계학과

2020.03 ~ 2026.08 (졸업예정)

SK Networks AI Camp 2171

2025.09 ~ 2026.03 · AI/ML 엔지니어링 집중 과정

CERTIFICATES

SQLD (SQL Developer)

한국데이터산업진흥원

ADsP (데이터분석 준전문가)

한국데이터산업진흥원

SKILLS

ML / LLM SERVING

vLLM LoRA LangGraph RAGAS Qdrant ChromaDB HyDE BM25+RRF Reranker OpenAI API MCP

BACKEND / INFRA

FastAPI Django PostgreSQL SQLite Docker

Nginx AWS (EC2, S3, RDS)

DATA / STATISTICS

Python pandas scikit-learn numpy GARCH(arch)

K-means PCA

SELECTED IMPACT

SEARCH QUALITY PYMATE

0.83 → 0.97

검색 병목을 다시 정의

RAGAS로 검색과 생성을 분리 측정해 병목을 찾고, ranking stack 재설계로 Precision과 Recall을 함께 개선했습니다.

SYSTEM RELIABILITY WORKFLOW AGENT

37% → 85%

실험을 API와 서빙으로 연결

FastAPI와 vLLM 기반으로 추론 API, 검증 레이어, 서빙 안정화까지 직접 연결해 실제 동작하는 시스템으로 만들었습니다.

SYSTEM OPTIMIZATION SEOUL CULTURE MAP

85% ↓

지표를 먼저 만들고 구조를 변경

정확도, 비용, latency 같은 지표를 먼저 정의하고 그 수치가 좋아지도록 라우팅과 캐싱 구조를 바꿔 왔습니다.

PROJECTS

PyMate — RAG 학습 튜터 챗봇

Team 4명 · 2026.01 ~ 02 · 6주

GitHub · 역할: 검색 품질 진단·retrieval 파이프라인 개선 담당 | 기여도 70%

팀 4명 기준, 검색 품질 진단·랭킹 구조·프로덕션 전환을 맡은 범위입니다.

"답변이 부정확하다"는 피드백을 감으로 해석하지 않고 검색·생성을 분리 측정해 검색 품질 병목을 embedding과 ranking 구조에서 찾았고, retrieval 재설계로 Precision 0.83→0.97(+14.4%)를 만들었습니다.

병목 재정의: 팀의 "LLM이 문제"라는 가설을 그대로 따르지 않고, RAGAS 4개 지표로 검색·생성을 분리 측정해 embedding 품질이 핵심 병목임을 확인했습니다.

검색 랭킹 재설계: 768D→3,072D embedding 교체, HyDE+BM25+RRF+cross-encoder reranker 구조로 ranking stack을 재구성해 **Precision 0.83→0.97(+14.4%), Recall 0.70→0.79(+12.7%)**를 달성했습니다.

실서비스 관점 보완: Flask MVP를 Django 기반 구조로 전환하고 Nginx→Gunicorn 502 이슈를 해결해 **배포 후 다운타임 0건**으로 안정화했습니다.

배운 점: 모델을 바꾸기 전에 검색과 생성 단계를 분리 측정해야 개선 속도가 빨라진다는 기준을 얻었습니다.

[Qdrant](#) [RAGAS](#) [HyDE](#) [BM25+RRF](#) [cross-encoder Reranker](#) [text-embedding-3-large](#) [Django](#) [AWS EC2](#)

WorkFlow Agent — 사내 업무 자동화 멀티 에이전트

Team 4명 · 2026.02 ~ 04 · 8주

GitHub · 역할: 품질 검증·서빙 안정화 담당 | 기여도 65%

팀 4명 기준, 검증 체계·서빙 안정화·LoRA 반복 실험을 맡은 범위입니다.

LLM이 "확신하면서 틀리는" 문제를 모델 성능이 아니라 시스템 품질 문제로 다시 정의하고, 온프레미스 서빙·검증·보정 구조를 설계해 정확도와 안정성을 함께 개선한 프로젝트입니다.

품질 문제 재정의: "정확도가 낮다"가 아니라 **과신하는 오답이 실행 단계로 넘어가는 구조**가 문제라고 정의하고, 모델 실험과 서비스 품질 관점을 분리해 접근했습니다.

검증 체계 설계: 키워드 매칭, 조항 존재 검증, 카테고리 제한, 일관성 모니터링의 4중 Guardrail과 confidence hard cap을 설계해 **고신뢰 오답 차단 구조**를 만들었습니다.

온프레미스 서빙 안정화: GPT API에 사내 데이터를 보낼 수 없는 제약 아래 RunPod A100에서 vLLM 기반 sLLM 서빙, JSON parsing fallback, LoRA 3회 반복 실험을 통해 **정확도 37%→85%, JSON 유효율 70%→97%**를 달성했습니다.

배운 점: LLM 품질은 모델 하나보다 검증 레이어와 실패 처리 구조까지 함께 설계해야 안정화된다는 점을 배웠습니다.

[vLLM](#) [LoRA](#) [LangGraph](#) [KoELECTRA](#) [Qdrant](#) [FastAPI](#) [RunPod A100](#) [Docker](#)

Seoul Culture Map — 서울 문화행사 탐색 시스템

Personal · 2026.03 ~ Present · Prototype

GitHub · [LangGraph](#) · [ChromaDB](#) · [FastAPI](#) · [scikit-learn](#) | 기여도 100%

문제 정의부터 라우팅 구조, API, UI, 배포까지 단독으로 진행한 개인 프로젝트입니다.

모든 요청에 동일한 파이프라인을 태우는 비효율을 문제로 보고, 의도 분류 라우팅과 메타데이터 필터링을 설계해 비용과 응답 속도를 동시에 개선했습니다.

라우팅 최적화: LangGraph 3-node(Classifier→DB/RAG/Chat 분기)로 단순 조회는 DB 직접 응답, 탐색형 질의만 RAG로 보내도록 구성해 **추정 요청 비용 85% 절감**을 만들었습니다.

검색 품질 보강: 서울시 공공 API 데이터를 ChromaDB에 임베딩하고 위치·날짜·카테고리 메타데이터 필터를 추가해 검색 결과의 문맥 적합도를 높였습니다.

제품 UX 구현: FastAPI SSE 스트리밍과 React 19/Leaflet UI를 붙여 **첫 토큰 응답 ~200ms** 수준의 탐색 경험을 구현했습니다.

배운 점: 모든 요청에 같은 파이프라인을 태우기보다 의도별로 나눠야 비용과 응답 속도를 함께 잡을 수 있었습니다.

[LangGraph](#) [ChromaDB](#) [FastAPI](#) [SSE](#) [SQLite](#) [React 19](#) [Leaflet](#) [scikit-learn](#) [GPT-4o-mini](#)

STATISTICS FOUNDATION

시계열 분석 · 변동성 모델링

5개 GARCH 모형 비교($R^2=0.89$), ADF-ARCH-LM 검정, AIC/BIC 기반 모형 선택. 암호화폐 변동성 분석에서 통계적 가설 검정→모형 선택→예측까지 전 과정 수행

차원 축소 · 군집분석

PCA(설명력 91.3%)로 고차원 데이터 압축 후 K-means 군집분석 적용. 가설 검정, 회귀분석, 다변량 분석 기반의 데이터 기반 의사결정 훈련

WHY DAANGN ML

왜 당근인가

당근은 "지역"이라는 강한 컨텍스트 위에서 검색 결과 품질, 키워드 의도 해석, 콘텐츠 필터링이 서비스 만족도에 바로 연결되는 환경이라고 생각합니다. 저는 PyMate에서 검색 품질을 분리 측정해 병목을 찾았고, Seoul Culture Map에서 의도 분류 라우팅으로 요청 유형을 나눠 본 경험이 있어 이런 문제 구조를 이해하고 배우는 데 강점이 있다고 생각합니다.

강점이 직무에 닿는 방식

제 경험은 검색 품질 평가와 ML 시스템 최적화 중심이라 learning-to-rank나 추천 모델을 직접 학습시킨 경험과는 거리가 있습니다. 다만 평가 기준을 먼저 세우고 단계별 병목을 분리 측정해 구조를 바꾸는 방식, 그리고 서빙 안정화와 품질 모니터링까지 함께 보는 시야는 당근 ML팀의 문제 해결 방식과 닿아 있다고 생각합니다.